Sets :→ (Uniqueness) ⇒ No duplicates :→

* Given an array of elements, remove the duplicates

arr = { 1, 2, 3, 5, 2, 1, 4, 3, 2, 6 }

O/P = { 1, 2, 3, 4, 5, 6 }

Set I
- → HashSet C
- → LinkedHashSet C [To maintain order]

SortedSet I
- → TreeSet C [Sorted order] (Ascending Order)

Collection
- → List
- → Queue
- → Set

Single values

≠ I Map → (Java) java.util

Multiple Values

<key, Value> pair

C HashMap

C LinkedHashMap

Hashtable (Hashing)

<Item, Price>

<Student, Marks>

<Equipment, Voltage>

I SortedMap → TreeMap C

Python:

dictionary

{1:1, 2:4, 3:9}

HashTable<K,V> { Array of lists } {Hashing} {LF} 0.75

```
HashTable<Integer, String> ht = new HashTable<>();
ht.put(106, "Anusheee");
ht.put(117, "Naveen");
ht.put(128, "Meghana");
ht.put(102, "Punith");
ht.put(100, "Sinchana");
ht.put(99, "Kavana");
ht.put(98, "Shathd Gowda");
```

10	98, Sha	(1)
9		
8		
7	106, Anushee	117, Naveen | 128, Meghana
6	(4)	(3) (2)
5		
4		
3	102, Punith	(5)
2		
1	100, Sinchana	(6)
0	99, Kavana	(7)

Hash Collision

Same index (multiple values)

Display:

Top to Bottom

Right to Left

[Initial size = 11] (106 % 11)

Store : Hashing: → Process of providing unique code (hashcode) to the data by JVM.

⇒ {10, 20, 30, 40, 50} ⇒ (10) (% 11) 11 prime numbers

{0 0 0 0 0 0} bucket

{10 | 9 | 8 | 7 | 6} index

index = number % capacity

= 10 % 11

= 10

7

5, 15, 25

% 10

5 5 5

(More hash collisions)